

VERIFICHE (NO STRUTTURA CODICE)

Classi

```
from dataclasses import dataclass

@dataclass
class Stipendi:
    tID:int
    tName:str
    salariTotali:int

    def __hash__(self):
        return hash(self.tID)

    def __eq__(self, other):
        return self.tID==other.tID

    def __str__(self):
        return f"{self.tName} : {self.salariTotali}€"
```

Riempire DropDown (accompagnato dal salvataggio della variabile, ricordarsi di salvarla nell'init, siamo nel controller)

```
def fillDropdown(self, allNodes):
    for n in allNodes:
        self._view._ddAeroportoP.options.append(
            ft.dropdown.Option(
                data = n, # Oggetto
                key = n.IATA_CODE, # Stringa che verrà visualizzata
                on_click = self._choiceDdPartenza # Dove viene salvato
            )
        )
```

```
def _choiceDdPartenza(self, e):
    self._coicePartenza = e.control.data
    print(f"Hai selezionato come aeroporto di partenza {self._coicePartenza}")
```

Verifica di parametro in input (siamo nel controller prima della creazione del grafo)

Verifica casella di testo vuota →

```
numeroScelto = self._view._txtCompagnieMinimo.value
if numeroScelto == "":
    self._view._txtResults.controls.clear()
    self._view._txtResults.controls.append(
```

```

        ft.Text("Inserire un valore numetico per numero minimo compagnie",
color="red"))
    self._view.update_page()
    return

```

Trasformazione in numero intero →

```

try:
    cMin = int(numeroScelto)
except ValueError:
    self._view._txtResults.controls.clear()
    self._view._txtResults.controls.append(ft.Text("Inserire un valore intero
per numero minimo compagnie",color = "red"))
    self._view.update_page()
    return

```

STRUTTURA CODICE

Nel model →

Init :

```

def __init__(self):
    self.grafo = nx.DiGraph()

    self.allAirports = DAO.getAllAirports()
    self._idMapAirports = {}
    for airport in self.allAirports:
        self._idMapAirports[airport.ID] = airport

```

Creazione del grafo :

```

def createGraph(self, anno):
    self.graph.clear()
    # Aggiungo nodi

    self.graph.add_nodes_from(self._squadre)
    print(f"Nodi: {len(self.graph.nodes)}, Edge: {len(self.graph.edges)}")
    # Aggiungo gli archi
    self.addEdges(anno)
    print(f"Nodi: {len(self.graph.nodes)}, Edge: {len(self.graph.edges)}")

def addEdges(self, anno):
    elencoSquadreSalari = DAO.getTeamsSalaries(anno)
    for i in range(len(elencoSquadreSalari)):
        for j in range(i+1, len(elencoSquadreSalari)):
            squadraA= self._idMapSquadre[elencoSquadreSalari[i].tID]
            squadraB= self._idMapSquadre[elencoSquadreSalari[j].tID]

```

```

        peso_arco = elencoSquadreSalari[i].salariTotali +
elencoSquadreSalari[j].salariTotali

        # Creazione dell'arco pesato
        if self.graph.has_node(squadraA) and self.graph.has_node(squadraB):
            self.graph.add_edge(squadraA, squadraB, weight=peso_arco)

def getGraphDetail(self):
    return len(self.graph.nodes), len(self.graph.edges)

```

Se gli edge li prendo da una query già fatta (nodoA, nodoB)

```

def addEdges(self, genereSelID):
    # Recuperiamo le coppie di oggetti Artista grazie alla tua idMap nel DAO
    coppie_artisti = DAO.getAllEdges(genereSelID, self._idMapArtisti)

    for u, v in coppie_artisti:
        # Recuperiamo la popolarità usando gli ID degli oggetti artista u e v
        # Usiamo .get(..., 0) nel caso un artista non abbia vendite registrate nel genere
        popU = self._popolarita.get(u.ArtistId, 0)
        popV = self._popolarita.get(v.ArtistId, 0)

        # Il peso dell'arco è la SOMMA delle rispettive popolarità
        peso_arco = popU + popV

        # Logica del testo per stabilire il VERSO del grafo orientato:
        if popU > popV:
            # Verso da A a B se la popolarità di A è maggiore della popolarità di B (u -> v)
            self.grafo.add_edge(u, v, weight=peso_arco)
        elif popV > popU:
            # Altrimenti, se popolarità di B > A, verso da v a u
            self.grafo.add_edge(v, u, weight=peso_arco)
        else:
            # In caso i nodi abbiano la stessa popolarità, aggiungiamo due archi in entrambi
            i versi
            self.grafo.add_edge(u, v, weight=peso_arco)
            self.grafo.add_edge(v, u, weight=peso_arco)

```

Se gli edge li prendo da query (NodoA, NodoB, peso)

```

def addEdges(self, IdStore, giornoUtente):

    tupleEdge = DAO.getEdges(IdStore, giornoUtente)

    for tupla in tupleEdge:
        self.grafo.add_edge(self._idMapOrdini[tupla[0]], self._idMapOrdini[tupla[1]], weight = tupla[2])

```

Ordinamento nodi (opzionale) :

```

def getAllNodes(self):
    nodes = list(self.grafo.nodes)
    nodes.sort(key = lambda x:x.IATA_CODE)
    return nodes

```

Trovare il cammino :

```
def getPath(self, v0, v1):

    path = nx.dijkstra_path(self.grafo, v0, v1, weight = None)
    # con nullo il wight indiachiamo a questo di algoritmo di fregarsene dei pesi

    # path = [v0, ----, v1]
    return path
```

Ha cammino? True False

```
def hasPath(self, v0, v1):
    # nx.has_path funziona correttamente sia su grafi orientati che non orientati
    return nx.has_path(self.grafo, v0, v1)
    # Una volta fatto questo metodo, se verificiamo questo, ha senso verificare un cammino
```

Vicini a un certo nodo

```
def getViciniNodo(self, nodoSource):

    elencoVicini = []
    nodiVicini = self.graph.neighbors(nodoSource)

    for i in nodiVicini:
        elencoVicini.append((i, self.graph[nodoSource][i]["weight"]))

    elencoVicini.sort(key=lambda x: x[1], reverse=True)

    return elencoVicini
```

Top 5 archi :

```
def get_top_5_archi(self):
    # Chiediamo gli archi passando direttamente data='weight'
    # Ogni arco nella lista sarà una tupla semplice: (u, v, peso)
    lista_archi = list(self.grafo.edges(data='weight'))

    # Ordiniamo la lista guardando l'elemento con indice 2 (il peso) in ordine decrescente
    lista_archi.sort(key=lambda x: x[2], reverse=True)

    # Restituiamo solo i primi 5
    return lista_archi[0:5]
```

Trovare il nodo con più archi adiacenti (Grado Massimo)

```
def get_nodo_top(self):
    max_vicini = -1
    nodo_top = None

    for nodo in self.grafo.nodes:
        # len(self.grafo[nodo]) restituisce il numero di vicini
        num_vicini = len(self.grafo[nodo])
        if num_vicini > max_vicini:
            max_vicini = num_vicini
            nodo_top = nodo
    return nodo_top, max_vicini
```

Ottenere i vicini di un nodo ordinati per peso dell'arco, Molti temi d'esame chiedono: *"Dato il nodo X inserito dall'utente, stampare i suoi vicini in ordine decrescente di peso dell'arco"*

```
def get_vicini_ordinati(self, nodo_scelto):
    vicini_pesati = []
    # Esploriamo i vicini del nodo scelto
    for vicino in self.grafo.neighbors(nodo_scelto):
        # Recuperiamo il peso dell'arco tra il nodo scelto e il vicino
        peso = self.grafo[nodo_scelto][vicino]['weight']
        vicini_pesati.append((vicino, peso))

    # Ordiniamo la lista in base al peso (indice 1 della tupla)
    vicini_pesati.sort(key=lambda x: x[1], reverse=True)
    return vicini_pesati
```

Calcolo del Grado di Bilancio (In-Degree vs Out-Degree), nei grafi orientati, oltre all'influenza (che usa i pesi), spesso chiedono semplicemente il bilancio tra il numero di archi entranti e uscenti (senza contare il peso).

```
def get_bilancio_archi(self):
    miglior_nodo = None
    max_bilancio = -999999

    for nodo in self.grafo.nodes:
        # self.grafo.in_degree(nodo) conta quanti archi entrano
        # self.grafo.out_degree(nodo) conta quanti archi escono
        entranti = self.grafo.in_degree(nodo)
        uscenti = self.grafo.out_degree(nodo)

        bilancio = uscenti - entranti

        if bilancio > max_bilancio:
            max_bilancio = bilancio
            miglior_nodo = nodo

    return miglior_nodo, max_bilancio
```

Trovare i Nodi Isolati, Un grande classico: *"Trovare tutti gli artisti (o nodi) che non hanno relazioni con nessun altro artista nel grafo"*. Un nodo è isolato se il suo grado totale è zero.

```
def get_nodi_isolati(self):
    isolati = []

    for nodo in self.grafo.nodes:
        # degree(nodo) restituisce la somma di archi entranti e uscenti
        if self.grafo.degree(nodo) == 0:
            isolati.append(nodo)

    return isolati
```

Verificare la presenza di un cammino diretto (Raggiungibilità), A volte il testo chiede: *"Dati due nodi selezionati dall'utente (Nodo A e Nodo B), verificare se esiste un cammino che li unisce e calcolarne la distanza minima (numero di archi)"*.

```
def verifica_percorso(self, nodo_A, nodo_B):
    # 1. Controlla se esiste un percorso tra A e B
    esiste_percorso = nx.has_path(self.grafo, nodo_A, nodo_B)

    if esiste_percorso:
        # 2. Trova la lista di nodi del cammino più breve (BFS/Dijkstra automatico)
        cammino_breve = nx.shortest_path(self.grafo, source=nodo_A, target=nodo_B)

        # 3. Trova la lunghezza (numero di archi, che è il numero di nodi - 1)
        lunghezza = len(cammino_breve) - 1
        return esiste_percorso, cammino_breve, lunghezza
    else:
        return esiste_percorso, [], 0
```

Calcolare la media dei pesi del grafo, Un'altra richiesta statistica insidiosa: *"Calcolare il peso medio di tutti gli archi presenti nel grafo"*.

```
def get_peso_medio_archi(self):
    if len(self.grafo.edges) == 0:
        return 0

    somma_pesi = 0
    # Cicliamo su tutti gli archi prendendo il peso
    for u, v, peso in self.grafo.edges(data='weight'):
        somma_pesi += peso

    peso_medio = somma_pesi / len(self.grafo.edges)
    return peso_medio
```

Trovare i "Vicini in Comune" (Shared Neighbors), Nei grafi non orientati, il testo potrebbe chiedere: *"Dati due artisti selezionati dall'utente, stampare quali altri artisti sono connessi a entrambi"*.

```
def get_vicini_comuni(self, nodo_A, nodo_B):
    # Spostiamo i vicini in due set (insiemi) di Python
    set_A = set(self.grafo.neighbors(nodo_A))
    set_B = set(self.grafo.neighbors(nodo_B))

    # L'operatore '&' trova l'intersezione (gli elementi in comune)
    comuni = list(set_A & set_B)
    return comuni
```

Hubs e Autorità: I Nodi con il Grado di Ingresso/Uscita Massimo, Mentre l'algoritmo visto prima calcolava la differenza (*uscenti - entranti*), a volte chiedono semplicemente: *"Trovare l'artista che riceve più archi (il più passivo/scelto) o quello che ne fa partire di più"*.

```
def get_top_in_out_degree(self):
    max_in = -1
    nodo_max_in = None

    max_out = -1
    nodo_max_out = None

    for nodo in self.grafo.nodes:
        # Grado di ingresso (quanti entrano)
        if self.grafo.in_degree(nodo) > max_in:
            max_in = self.grafo.in_degree(nodo)
            nodo_max_in = nodo

        # Grado di uscita (quanti escono)
        if self.grafo.out_degree(nodo) > max_out:
            max_out = self.grafo.out_degree(nodo)
            nodo_max_out = nodo

    return nodo_max_in, max_in, nodo_max_out, max_out
```

Trovare i "Cicli" nel Grafo (Solo se Orientato), Raramente, ma è successo, la traccia chiede di verificare se ci sono situazioni di preferenza circolare (es. A preferisce B, B preferisce C, C preferisce A). NetworkX lo fa in un'unica riga.

```
def rileva_cicli(self):
    # Controlla se esiste almeno un ciclo nel grafo orientato
    # (restituisce True se il grafo NON è un albero diretto/DAG)
    ha_cicli = not nx.is_directed_acyclic_graph(self.grafo)
```

```

elenco_cicli = []
if ha_cicli:
    # Estrae una lista di liste, dove ogni sotto-lista è un ciclo di nodi
    elenco_cicli = list(nx.simple_cycles(self.grafo))

return ha_cicli, elenco_cicli

```

COMPONENTI CONNESSE

```

def get_connected_components_details(self):
    """
    Ritorna il numero di componenti connesse e i nodi della componente più grande.
    Funziona nativamente solo su grafi NON orientati.
    Se il grafo è orientato, si usa .to_undirected() per trovare la connettività
    debole.
    """
    if self.grafo.is_directed():
        grafo_non_orientato = self.grafo.to_undirected()
    else:
        grafo_non_orientato = self.grafo

    # Estrae tutte le componenti come insiemi di nodi
    componenti = list(nx.connected_components(grafo_non_orientato))
    num_componenti = len(componenti)

    if num_componenti == 0:
        return 0, []

    # Trova la componente con più nodi
    componente_maggiore = max(componenti, key=len)
    lista_nodi_maggiore = list(componente_maggiore)

    return num_componenti, lista_nodi_maggiore

```

Alla pressione del tasto Stampa dettagli, il programma dovrà identificare la componente connessa di dimensione maggiore, e stamparne tutti i nodi, ordinati in senso decrescente di peso minimo degli archi incidenti.

```

def get_dettagli_componente_maggiore(grafo_f1: nx.Graph):
    """
    Identifica la componente connessa di dimensione maggiore e restituisce
    i nodi ordinati in senso decrescente rispetto al peso minimo degli archi incidenti.
    """
    # Gestione caso bordo: grafo vuoto
    if not grafo_f1 or grafo_f1.number_of_nodes() == 0:
        return []

    # 1. Identificare tutte le componenti connesse
    # nx.connected_components restituisce insiemi (set) di nodi
    componenti = list(nx.connected_components(grafo_f1))

    # 2. Selezionare la componente connessa di dimensione maggiore
    # Usiamo len come chiave per trovare l'insieme con più nodi

```



```
componente_maggiore = max(componenti, key=len)

risultati = []

# 3. Calcolare il peso minimo degli archi incidenti per ciascun nodo della componente
for nodo in componente_maggiore:
    # Recuperiamo tutti gli archi incidenti su questo specifico nodo
    archi_incidenti = grafo_f1.edges(nodo, data=True)

    if not archi_incidenti:
        # Caso in cui il nodo sia totalmente isolato (componente di dimensione 1)
        peso_minimo = 0
    else:
        # Estraiamo i pesi di tutti gli archi incidenti
        pesi = [dati_arco['weight'] for _, _, dati_arco in archi_incidenti]
        peso_minimo = min(pesi)

    risultati.append((nodo, peso_minimo))

# 4. Ordinare in senso DECRESCENTE rispetto al peso minimo trovato
# x[1] corrisponde al peso_minimo nella tupla (nodo, peso_minimo)
risultati.sort(key=lambda x: x[1], reverse=True)

return risultati
```

Nel Controller →

Init :

```
def __init__(self, view, model):
    # the view, with the graphical elements of the UI
    self._view = view
    # the model, which implements the logic of the program and holds the data
    self._model = model

    self._coicePartenza = None
    self._coiceArrivo = None
```

Creazione Grafo (nella funzione apposita dopo opportune verifiche) :

```
self._model.creaGrafo(cMin)
numNodes, numEdges = self._model.getGraphDetails()

allNodes = self._model.getAllNodes()

self._view._txtResults.controls.clear()
self._view._txtResults.controls.append(ft.Text("Grafo correttamente creato",
color="green"))
self._view._txtResults.controls.append(
    ft.Text(f"Il grafo contiene {numNodes} nodi e {numEdges} archi", color="green"))
self._view.update_page()
```

La verifica dei cammini va fatta qui ma la logica nel model

Stampare i primi 5 archi

```
# 3. Stampa i top 5 archi
self._view.txt_result.controls.append(ft.Text("\nI 5 archi con peso maggiore:"))
top_5 = self._model.get_top_5_archi()
for u, v, data in top_5:
    self._view.txt_result.controls.append(ft.Text(f"{u.Name} -> {v.Name} | Peso:
{data['weight']}"))

self._view.update_page()
```